73. Введение в тестирование кода

Итак, вы написали программу, она компилируется, и даже работает! Что дальше? Есть несколько вариантов.

Оглавление:

1. **Зачем выполнять тестирование?**
2. **Выполнение неофициального тестирования**
   - **Совет №1**
   - **Совет №2**
   - **Совет №3**
   - **Совет №4**
   - **Совет №5**
3. **Сохранение ваших тестов**
4. **Автоматизация тестирования**
5. **Тест**

Зачем выполнять тестирование?

Если вы написали программу, чтобы её один раз запустить и забыть, то дальше начнёт расти не нужно. Здесь не стоять можно, что ваша программа может некорректно работать в некоторых случаях: если при первом запуске она работает, так как вы и ожидали, и если вы дальше запускать и использовать её не планируете, тогда всё — фиговы.

Если ваша программа выполняется несколько раз и может использовать какой-нибудь прочитанный файл (или switch), не прочитанный вводимые данные и выдавать правильный результат, тогда тоже фиговы. В этом случае вы уже протестировали все программы, запустив её один раз и сверив результат.

Но если вы написали программу, которую собираетесь запускать много раз и которая имеет цикл и условные ветвления, принимает пользовательский ввод, то здесь уже немного по-другому. Возможно, вы написали функцию, которую хотите повторно использовать в другом проекте. Возможно, вы даже намереваетесь распространять эту программу в дальнейшем. В таком случае вам действительно нужно будет проверить, как ваша программа работает в самых разных условиях.

Только потому, что она корректно выполнилась с одними значениями пользовательского ввода, совсем не значит, что она будет работать корректно и с другими значениями.

Тестирование программного обеспечения — это процесс определения работоспособности программного обеспечения согласно ожиданиям разработчиков.

Прежде чем мы будем говорить о некоторых практических способах тестирования вашего кода, давайте поговорим о том, почему компьютеры тестирование может быть сложным. Например, рассмотрим следующую программу:

```
// Prompt user for two numbers
int x = get_int("x: ");
int y = get_int("y: ");

void compare(int a, int b)
{
    if (a < b)
    {
        printf("%d is less than %d\n", a, b);
    }
    else if (a > b)
    {
        printf("%d is more than %d\n", a, b);
    }
    else
    {
        printf("%d is equal to %d\n", a, b);
    }
}

int main()
{
    char *msg = "Enter a number: ";
    int a, b;
    while (true)
    {
        int x = get_int("%s", msg);
        int y = get_int("Enter another number: ");
        if (x == y)
        {
            break;
        }
        msg = "Try again: ";
        compare(x, b);
    }
}
```

Учитывая 4 byte/long int или 8 long long int диапазон значений, для которого все возможные значения нам потребуется 18 446 744 073 709 551 616 (~ 18 квинтиллионов) для каждого из параметров. Понятно, что это абсурд.

Каждый раз, когда мы запрашиваем пользовательский ввод или используем условное ветвление в программе — мы увеличиваем количество потенциальных способов выполнения нашей программы. Для всех программ, какие простейшие, потенциал каждое изображение вводных данных, для ещё и вручную — как то не логично, вам не кажется?

Сейчас ваш ноутбук должен подсказывать вам, что для того, чтобы убедиться в полной работоспособности программы, необходимо протестировать 18 квинтиллионов вариантов. Если ваш компьютер может выполнять 1 миллиард операций в секунду, выполнение всех вариантов потребует 1 с, а раз уже стал при одной значений 8 и 2, то под данным корректно работать и с любыми другими значениями 2 и 2, где 2 и 2, 3 и 3 и т.д. Учитывая это, становится очевидным, что для тестирования программы нам потребуется запустить не всего лишь три раза (по одному для каждого случая: x < y, x > y, x == y), а чтобы убедиться, что она работает корректно. Есть и другие причины, позволяющие упростить процесс тестирования кода.

Программы тестирования можно долго рассказывать, но так как это тема не является специфической именно для языка C++, то мы будем придерживаться краткого и понятного изложения материала для начинающего разработчика, который тестирует свой собственный код.

Выполнение неофициального тестирования

Большинство разработчиков тестирование **неофициальное**, когда пишут свои программы. После написания части кода формируют, какая или какого-либо другого кода ещё разработчик пишет новой кода для проверки только что добавленного кода, а если тот пройдёт успешно, продолжают работать дальше над этого теста. Например, для следующей функции isclosetwo() мы можем написать следующий код для проверки:

```
int isclosetwo(char c)
{
    switch (c)
    {
        case 'a':
        case 'b':
        case 'c':
        case 'd':
        case 'e':
        case 'f':
        case '0':
        case '1':
        case '2':
        case '3':
        case '4':
        case '5':
        case '6':
        case '7':
        case '8':
        case '9':
        return true;
        default:
        return false;
    }
}

int main()
{
    char char = isclosetwo('a');// ожидаемый положит знак, результатом которого должно быть 0
    char char = isclosetwo('1');// ожидаемый положит знак, результатом которого должно быть 1
}
```

Если при выполнении программы мы получим 1 и 0, тогда всё хорошо. Вы знаете, что ваша функция работает, поэтому можно сделать временный тестовый код и продолжить процесс программирования.

Совет №1: Пишите свою программу по частям: в небольшие, чётко определенных куски (функции)

Возьмите, к примеру, автопроизводителя, который создает автомобили. Как вы думаете, что из следующего он делает?

Ответ: он не создает! Производитель не создает целый автомобиль сразу. Вместо этого производитель строит отдельные части автомобиля, которые затем собирают вместе. Производитель интересует все возможные проверки, чтобы убедиться, что интересующие детали в порядке. В конце, перед транспортной, производителя тщательный тест работоспособности всего автомобиля.

Создает автомобиль из всех компонентов без какой-либо предварительной проверки (за одно присест). Затем, в конце, проводится первая и окончательная тестирование работоспособности уже собранного автомобиля.

Не кажется вам, что более правильным является первый вариант? И вот же большинство начинающих программистов пишут свой код в соответствии со вторым вариантом!

Во втором случае, если какая-либо часть не застрой большинство будет работать, неправильно, то именно придётся проистечь диагностику всего автомобиля, чтобы определить, что пошло не так — проблема может находиться где угодно. Например, автомобиль может не завестись из-за неисправной свечи зажигания, аккумулятора, топливного насоса или чего-то еще. Это приведет к потере большого количества потраченного впустую времени (плюс потенциальные затраты на ремонт). Если вскоре проблема будет найдена, поскольким могут быть катастрофические изменения в целой части автомобиля могут приместа с «эффект бабочки» — серьезные изменения в другом части автомобиля. Например, сгоревшая маленькой топливной насос может примести к изменению расстояния, что приведет к разрушению корпуса автомобиля. В конечном счете производитель передвигаешь большую часть дель просто чтобы исправить то, что изначально было небольшой проблемой.

В первом случае автопроизводитель проверяет все детали по мере поступления. Если какой-либо из компонентов оказался бракованным, то механик сразу бросающий, и как проблема и где же искать. Почти не интересуются и автомобиль, пока не будет успешно протестирован. К тому времени, когда они уже собирут весь автомобиль, у них будет разумное уверенность в его работоспособности — и какая сладкая, вот его чаще были успешно протестированы. Все же есть вероятность, что что-то может пойти не так при соединении всех частей, но по сравнению со вторым вариантом — это очень малая вероятность, с которой и не следует сильно волноваться.

Вышеупомянутый аналогия справедлива и для программистов, хотя, по некоторым причинам, можно часто этого не осознают. Пишите свою программу небольшими частями, а затем сразу же начинайте тестировать. Таким образом, если вы допустили ошибку, вы будете знать, что она находится в небольшом количестве кода, который вы только что написали! Изменения. А что, в свою очередь, означает, что площадь поиска сильно меньше, а времени на отладку будет гораздо меньше в конечном.

Правило: Тестируйте свой код и всегда тестируйте все нетривиальные функции, которые вы пишете.

Совет №2: Нацеливайтесь на 100%-ное покрытие кода

Термин **«покрытие кода»** относится к количеству исходного кода программы, который был задействован во время выполнения. Например, если ваша программа показывает пользователю один из ответов, а нажмешь на кнопку стоит отмечать.

Покрытие стейтментов — это процент стейтментов в вашем коде, которые были задействованы во время выполнения тестирования. Например:

```
int isclosetwo(char c)
{
    bool d = 0;
    switch (c)
    {
        case 'a':
        case 'b':
        case 'c':
        case 'd':
        case 'e':
        case 'f':
        case '0':
        case '1':
        case '2':
        case '3':
        case '4':
        case '5':
        case '6':
        case '7':
        case '8':
        case '9':
        return true;// стейтмент №1
        default:
        return false;// стейтмент №2
    }
}
```

Здесь протестированы два разных пути выполнения. Если каждый либо из компонентов оказался бракованным, то механик сразу бросающий, и как проблема и где же искать. Почти не интересуются и автомобиль, пока не будет успешно протестирован. К тому времени, когда они уже собирут весь автомобиль, у них будет разумное уверенность в его работоспособности — и какая сладкая, вот его чаще были успешно протестированы. Все же есть вероятность, что что-то может пойти не так при соединении всех частей, но по сравнению со вторым вариантом — это очень малая вероятность, с которой и не следует сильно волноваться.

Вышеупомянутый аналогия справедлива и для программистов, хотя, по некоторым причинам, можно часто этого не осознают. Пишите свою программу небольшими частями, а затем сразу же начинайте тестировать. Таким образом, если вы допустили ошибку, вы будете знать, что она находится в небольшом количестве кода, который вы только что написали! изменения. А что, в свою очередь, означает, что площадь поиска сильно меньше, а времени на отладку будет гораздо меньше в конечном.

Правило: Пишите свою программу по частям: в небольшие, чётко определенных куски (функции).

Совет №3: Нацеливайтесь на 100%-ное покрытие ветвлений

Термин **«покрытие ветвлений»** относится к процентам ветвлений, которые были выполнены в каждом случае (положительном и отрицательном) условия. Оператор switch может иметь много ветвлений. Оператор if имеет два ветвления: случай true и случай false даже если нет оператора else. Например:

```
int isclosetwo(char c)
{
    bool d = 0;
    if (d == 0)
    {
        return 0;
    }
    else
    {
        return 1;
    }
}
```

Предыдущий вызов bool(0, 0) дал нам 100%-ный охват стейтментов и ветвлений true. Но это всего лишь 50%-ный охват ветвлений. Нам нужно еще один вызов — bool(0, 1), чтобы получить 100%-ный охват ветвлений.

В функции isclosetwo() нужно для вызова (например, isclosetwo('a') и isclosetwo('1')) чтобы убедиться в 100%-ном охвате ветвлений (все случаи, которые находятся в switch, тестировать не обязательно, если требуется одна — требуется и другие):

```
int isclosetwo(char c)
{
    switch (c)
    {
        case 'a':
        case 'b':
        case 'c':
        case 'd':
        case 'e':
        case 'f':
        case '0':
        case '1':
        case '2':
        case '3':
        case '4':
        case '5':
        case '6':
        case '7':
        case '8':
        case '9':
        return true;
        default:
        return false;
    }
}

int main()
{
    char char = isclosetwo('a');// ожидаемый положит знак, результатом которого должно быть 0
    char char = isclosetwo('1');// ожидаемый положит знак, результатом которого должно быть 1
}
```

Перехватим функции сравнения из первого примера данного раза:

```
void compare(int a, int b)
{
    if (a < b)
    {
        printf("%d is less than %d\n", a, b);// стейтмент №1
    }
    else if (a > b)
    {
        printf("%d is more than %d\n", a, b);// стейтмент №2
    }
    else
    {
        printf("%d is equal to %d\n", a, b);// стейтмент №3
    }
}
```

Здесь необходимы 3 вызова функции, чтобы получить 100%-ный охват ветвлений:

- compare(1,0) — проверяет вариант true для первого оператора if.
- compare(0, 1) — проверяет вариант false для первого оператора if и вариант true для второго оператора if (else if).
- compare(0, 0) — проверяет вариант false для второго оператора if и выполняет последний else.

Таким образом, мы можем сказать, что эту функцию можно протестировать с помощью всего трех вызовов функции (а не 18 квинтиллионов раз).

Правило: Тестируйте каждый случай ветвления и каждый случай условия.

Совет №4: Нацеливайтесь на 100%-ное покрытие циклов

Покрытие циклов (информальное называемый **«охват 0, 1, 2»**) показывает, что если у вас есть цикл в коде, то, чтобы убедиться в его работоспособности, нужно его выполнить 0, 1 и 2 раза. Если он работает корректно на второй итерации, то должно работать корректно и для всех последующих итераций (3, 4, 10, 100 и т.д.). Например:

```
int main(int timesToPrint)
{
    for (int i=0; i<timesToPrint; i++)
        int char = "Test";
}
```

Чтобы протестировать цикл внутри функции, нам придется вызвать его 3 раза:

- main(0) - чтобы проверить случай нулевой итерации
- main(1) для проверки итерации №1 и main(2) для проверки итерации №2.
- Если main(2) работает, тогда и main(3) будет работать (где x > 2).

Правило: Используйте охват 0, 1, 2х для проверки циклов на корректную работу с разным количеством итераций.

Совет №5: Убедитесь, что вы тестируете разные типы вводу

Когда вы пишете функции, которые принимают параметры или пользовательский ввод, то посмотрите, что происходит с разными типами ввода. Например, если в написали функции выполняется квадратного корня из целого числа, то какие значения вы ожидаете в качестве параметров? Вы ожидаете только обычные значения, например, 2 и 16, такие были бы нормально протестированы и 4 и какие-нибудь отрицательные числа.

Вот только основные ожидаемые результаты для тестирования разных типов ввода:

- Для целых чисел обычных, что мы проверили, как ваша функция обрабатывает 2, отрицательные и положительные значения. При наличии большого количества параметров вы также должны проверить каждый возможный **перекрывания**.
- Для чисел типа с плавающей запятой обычных, что мы рассмотрели выше, как ваша функция обрабатывает значения, которые имеют нетипичности значения, которые имеют большинственные ожидаемых. Хорошие тестовые значения — это 0, 1 и -0.0. Для проверки чисел, которые имеют больше ожидаемых и 0, 1 и -0.0 (для проверки чисел, которые имеют меньшие ожидаемых).
- Для строк обычных, что вы рассмотрели выше, как ваша функция обрабатывает пустую строку, строку с допустимыми значениями, строку с проблемами и строку, содержащую которой является пробелом.

Правило: Тестируйте разные типы ввода, чтобы убедиться, что ваш код корректно обрабатывает.

Сохранение ваших тестов

Хотя комплексное тесты и последующие их удаление — достаточно хороший вариант для быстрого и временного тестирования, но для кода, который вы намереваетесь повторно использовать или модифицировать в будущем, имеет смысл сохранить эти тесты. Например, вместо удаления вашего временного теста, вы можете превратить его в функцию test():

```
int isclosetwo(char c)
{
    switch (c)
    {
        case 'a':
        case 'b':
        case 'c':
        case 'd':
        case 'e':
        case 'f':
        case '0':
        case '1':
        case '2':
        case '3':
        case '4':
        case '5':
        case '6':
        case '7':
        case '8':
        case '9':
        return true;
        default:
        return false;
    }
}

void test()
{
    char char = isclosetwo('a');// ожидаемый положит знак, результатом которого должно быть 0
    char char = isclosetwo('1');// ожидаемый положит знак, результатом которого должно быть 1
}

int main()
{
    test();
}
```

Автоматизация тестирования

Одна из проблем с вышеупомянутой тестовой функцией заключается в том, что вам придется вручную проверять результаты теста. А можно сделать лучше — добавить к тесту правильные ожидаемые результаты, которые должны получиться при успешном тестировании.

```
int isclosetwo(char c)
{
    switch (c)
    {
        case 'a':
        case 'b':
        case 'c':
        case 'd':
        case 'e':
        case 'f':
        case '0':
        case '1':
        case '2':
        case '3':
        case '4':
        case '5':
        case '6':
        case '7':
        case '8':
        case '9':
        return true;
        default:
        return false;
    }
}

// Проверяем новый тест, который не был прежде, чем мы сделали тест первым делом
void test()
{
    char char = isclosetwo('a');// ожидаемый положит знак, результатом которого должно быть 0
    char char = isclosetwo('1');// ожидаемый положит знак, результатом которого должно быть 1
    char char = isclosetwo('2');// ожидаемый положит знак, результатом которого должно быть 1
}

int main()
{
    test();
}
```

Теперь мы можете вызывать test() в любое время и функция сама всё сделает за вас.

Тест

Задание №1

Когда вы должны начинать тестировать свой код?

Задание №2

Сколько тестов потребуется для минимального подтверждения работоспособности следующей функции?

```
bool isclosetwo(char c, bool pitomec)
{
    switch (c)
    {
        case 'a':
        case 'b':
        case 'c':
        case 'd':
        case 'e':
        case 'f':
        case '0':
        case '1':
        case '2':
        case '3':
        case '4':
        case '5':
        case '6':
        case '7':
        case '8':
        case '9':
        return true;
        default:
        return false;
    }
}

// Проверяем новый тест, который не был прежде чем мы сделали тест первым делом
void test()
{
    char char = isclosetwo('a', true);// true, false
}

int main()
{
    test();
}
```

Ответ №1

Сразу, как только напишете нетривиальную функцию.

Ответ №2

Четырех тестов будет достаточно:

- Один для проверки случая `a/c/i/o/u`.
- Один для проверки случая по умолчанию.
- Один для тестирования `isclosetwo('y', true)`.
- И один для тестирования `isclosetwo('y', false)`.